

Machine Learning in Production

MLOps Engineering — A 12-Week Industry-Ready Program

Duration: 12 Weeks (3 Months)	Total Volume: 132 Contact Hours	Format: Theory + Hands-on Labs + Capstone	Level: Production MLOps Engineering
-------------------------------	---------------------------------	---	-------------------------------------

Program Overview

This program takes students from data engineering fundamentals through experiment tracking, deployment, monitoring, and automated retraining, into managed cloud ML platforms. Rather than a set of disconnected exercises, the course follows a **single progressive build**: each week adds a new capability to a pipeline the student carries forward, so that by the end of the program every student has a working, monitored, self-retraining ML system deployed on the cloud.

The Industry Learning Cycle: Every topic follows a battle-tested 7-stage learning cycle: **Concept** → **Architecture** → **Demonstration** → **Build** → **Break** → **Debug** → **Evaluate**. This ensures students don't just train a model in a notebook — they understand why production ML systems fail differently, and how to keep one alive in mission-critical environments.

Program Structure & Curriculum Breakdown

Component	Hours	Purpose & Focus
Theory / Concepts	66 h	Understand why production ML fails differently from notebook ML
Hands-on Labs	44 h	Build and test individual pipeline and infrastructure components
Projects / Capstone	18 h	Integrate into one deployed, monitored, self-retraining system
Evaluation / Defense	4 h	Defend architecture choices and operational tradeoffs

Technical Prerequisites

Required (Pre-Course)	Not Required (Covered in Program)
• Python (intermediate — functions, OOP basics) • SQL fundamentals • Linux & Git basics • Basic statistics (self-paced, before Week 1) • Familiarity with any ML library (sklearn/pandas)	• Prior deployment or DevOps experience • Cloud certification • Distributed systems expertise • Deep learning research background

* Note: Students without the required background can take an optional bridge module in Python, SQL, and Linux/Git before the program begins.

Week-by-Week Curriculum (Weeks 1 to 12)

Week	Module & Title	Key Concepts	Hands-on Practical Work
Week 1	Python & Engineering Practices for ML	Writing production-grade Python (packaging, testing, typing), why notebook code fails in production, reproducibility basics, config management.	Convert a notebook prototype into a packaged, tested Python module with a CI-ready project structure.
Week 2	Data Engineering Fundamentals: OLTP vs OLAP	OLTP vs OLAP workloads, why ML needs both, data warehouse vs data lake, columnar storage, intro to DuckDB.	Model a transactional (OLTP) dataset and rebuild it as an analytical (OLAP) schema; run benchmarked analytical queries in DuckDB.
Week 3	ETL Pipelines & Orchestration	ETL vs ELT, pipeline design patterns, scheduling, idempotency, backfills, orchestration concepts (DAGs).	Build a scheduled, idempotent ETL pipeline (extract → transform → load) orchestrated with Airflow/Prefect.
Week 4	Streaming Data with Kafka	Batch vs streaming, Kafka architecture (producers, consumers, topics, partitions), when ML systems need streaming.	Build a Kafka producer/consumer pair simulating real-time events feeding a feature pipeline.

Week 5	Data Labelling & Data Quality	Labelling strategies (manual, weak supervision, active learning), labelling tools (Label Studio, CVAT, Prodigy), inter-annotator agreement, schema/drift validation at ingestion.	Set up a labelling workflow in Label Studio; add automated data-quality checks (Great Expectations) into the Week 3 pipeline.
Week 6	Data Versioning & Feature Stores	Why 'which data trained this model' is a production requirement, DVC/lakeFS concepts, feature stores and train/serve skew.	Version the pipeline's datasets with DVC; build a simple feature store serving consistent features for training and inference.
Week 7	ML Algorithms for Production	Choosing algorithms for production constraints (latency, interpretability, retraining cost) — linear/tree-based models, ensembles, when not to use deep learning.	Build a production-style training pipeline (scikit-learn/XGBoost) with a reusable feature-engineering step, trained on the Week 6 feature store.
Week 8	Experiment Tracking & Model Versioning	Experiment tracking (MLflow/W&B;), model registries, model versioning and lineage, reproducible training runs.	Instrument the Week 7 pipeline with MLflow tracking; register the best model with full lineage back to its data version.
Week 9	Model Deployment	Batch vs real-time serving, REST/gRPC model serving, Docker for ML, basic Kubernetes concepts, shadow/canary deployment patterns.	Containerize the registered model behind a FastAPI serving endpoint; deploy with a canary rollout strategy.
Week 10	Monitoring in Production	Data drift vs concept drift, model performance decay, monitoring metrics (latency, throughput, prediction distribution), alerting, ML observability.	Instrument the Week 9 service with drift detection (Evidently AI) and an alerting dashboard.
Week 11	Retraining Pipelines & CI/CD/CT	Continuous training (CT) triggers, automated retraining pipelines, CI/CD for ML (testing data and models, not just code), rollback strategies.	Build an automated retraining pipeline triggered by Week 10 drift alerts, with a CI/CD gate validating the new model before promotion.
Week 12	Cloud ML Services & Capstone	Managed ML platforms — AWS SageMaker, Azure ML, GCP Vertex AI — where to use managed vs self-hosted, cost/tradeoff comparison across the three.	Deploy the full pipeline (or a component) on one managed cloud platform. Capstone: integrate Weeks 1–11 into one deployed, monitored, auto-retraining system on a real dataset — presented and defended.

Capstone Project — Deployed, Monitored, Auto-Retraining ML System

Students synthesize everything learned into an integrated, end-to-end production ML system running on a real dataset:

- **Full Ingestion & Storage Pipeline:** Raw data ingestion via scheduled ETL and streaming Kafka pipelines, validated and stored in OLAP/DuckDB columnar schemas.
- **Data Quality & Versioning:** Automated data quality validations using Great Expectations, with data versions and lineages pinned via DVC.
- **Model Registry & Lineage:** Complete experiment tracking with MLflow, registering models with full lineage from raw data version to trained weights.
- **Containerized Serving & Canary Rollout:** FastAPI microservice containerized in Docker, with canary/shadow routing mechanisms.
- **Real-Time Drift Observability:** Continuous monitoring for data drift and concept drift using Evidently AI with live alert channels.
- **Automated Retraining & CI/CD/CT:** Drift-triggered retraining jobs with automated quality-gate checks before model promotion.
- **Cloud Deployment:** Deployed on at least one major managed cloud platform (AWS SageMaker, Azure ML, or GCP Vertex AI).
- **Technical Defense:** A comprehensive technical report defending architecture decisions and operational trade-offs.

Graduate Learning Outcomes

By the conclusion of this program, students will be verified capable to:

- ✓ Engineer production-grade data pipelines, both batch and streaming
- ✓ Distinguish OLTP and OLAP workloads and use DuckDB for analytical queries
- ✓ Set up data labelling and automated data-quality workflows
- ✓ Version data and models with full lineage using DVC and a model registry
- ✓ Choose and train production-appropriate ML algorithms

- ✓ Track experiments and manage models through a registry (MLflow/W&B;)
- ✓ Deploy models via containerized APIs with safe rollout patterns (canary/shadow)
- ✓ Monitor production models for data drift and concept drift
- ✓ Build automated retraining and CI/CD/CT pipelines with rollback safety
- ✓ Deploy and compare ML workloads on AWS SageMaker, Azure ML, and GCP Vertex AI